theorem

Vera: Weakest Preconditions for TICL over Rust

From Imp Structural Lemmas to Rust VCGen Rules

Lef Ioannidis

2026

Background: TICL over Imp

TICL Structural Lemmas for Imp

Core idea (from the TICL paper)
For a small imperative language `StImp` with mutable state, the entailment $[t,\ m \Vdash_L\ \varphi]_S$ is proved structurally via inference rules on the syntax of t.

Key rules from `imp-lemmas.tex`:

- **Seq**$_S$AU$_L$: sequential composition decomposes into bind
- **If**$_S$AU$_L$: conditional branches both satisfy the obligation
- **While**$_S$AG: infinite loop with coinductive invariant $\mathcal{R}$
- **While**$_S$AU$_L$: terminating loop with ranking function f

Goal: Lift these to Rust syntax $\rightarrow$ define `wp` for Vera.

Notation

$$\sigma \in \text{State (mutable references, heap)}$$

$$[t,\ \sigma \Vdash_L\ \varphi] \triangleq \text{“Rust term } t \text{ from state } \sigma \text{ satisfies } \varphi\text{”}$$

$$\mathtt{wp}(t,\ \varphi) \triangleq \text{weakest precondition on } \sigma \text{ s.t. } [t,\ \sigma \Vdash_L\ \varphi]$$

Temporal formulas

- $\varphi ::= \mathtt{now}\ P \mid \mathtt{AG}\ \varphi \mid \varphi\ \mathtt{AU}\ \varphi' \mid \varphi\ \mathtt{AN}\ \varphi' \mid \varphi \wedge \varphi'$
- Sugar: $\mathtt{AF}\ \varphi' = \top\ \mathtt{AU}\ \varphi', \quad \mathtt{AX}\ \varphi' = \top\ \mathtt{AN}\ \varphi'$
- $\mathtt{now}\ P$ where $P : \sigma \to \mathbb{P}$ — state predicate on current state
- $\mathtt{done}\ Q$ where $Q : \tau \to \sigma \to \mathbb{P}$ — predicate on return value and final state

Inference Rules (Single-Process)

Assignment

Rust: $*x = e$

$$\frac{\begin{array}{c}[e,\ \sigma \Vdash_R\ \varphi\ \texttt{AU done}\ \mathcal{R}] \\ \forall\ v, \sigma',\ \mathcal{R}\ v\ \sigma' \to [\texttt{skip},\ \sigma'[x \mapsto v] \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']\end{array}}{[*x = e,\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']}\ \textsc{AssignAU}_L$$

Evaluating e may take steps; φ must hold along the path.

Sequential Composition (let-binding)

Rust: $\texttt{let}\ x = t;\ u$

$$\frac{[t,\ \sigma \Vdash_R\ \varphi\ \texttt{AU done}\ \mathcal{R}] \qquad \forall\ x, \sigma',\ \mathcal{R}\ x\ \sigma' \to [u[x],\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']}{[\texttt{let}\ x = t;\ u,\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']}\ \textsc{LetAU}_L$$

Direct lift of **BindAU**$_L$ / **Seq**$_S$AU$_L$ from TICL.

Conditional

Rust: `if` c $\{t\}$ `else` $\{u\}$

$$\frac{\begin{array}{l}[c,\ \sigma \Vdash_R\ \varphi\ \texttt{AU}\ \texttt{done}_{=}\ (b, \sigma')] \\ \begin{cases}[t,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi'], & \text{if } b \\ [u,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi'], & \text{otherwise}\end{cases}\end{array}}{[\texttt{if}\ c\ \{t\}\ \texttt{else}\ \{u\},\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']}\ \textsc{IfAU}_L$$

Evaluating c may take multiple steps; φ must hold along the path.

While Loop (AU – terminating)

Rust: `while` c $\{t\}$ **with** `invariant` $\mathcal{R}$, `decreases` f

$$\frac{\begin{array}{l}\forall\ \sigma,\ \mathcal{R}\ \sigma \to [c,\ \sigma \Vdash_R\ \varphi\ \texttt{AU done}_{=}\ (b, \sigma')] \\ \qquad \mathcal{R}\ \sigma \\ \qquad \wedge \begin{cases} [t,\ \sigma' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi'] \vee & \\ \quad [t,\ \sigma' \Vdash_R\ \varphi\ \texttt{AU done}\ (\lambda\ \sigma''.\ \mathcal{R}\ \sigma'' \wedge f\ \sigma'' < f\ \sigma)], & \text{if } b \\ \varphi'(\sigma'), & \text{otherwise} \end{cases}\end{array}}{[\texttt{while}\ c\ \{t\},\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']}\ \textsc{WhileAU}_L$$

While Loop (AG – infinite)

Rust: `while` c $\{t\}$ **with** `invariant` $\mathcal{R}$ **(no** `decreases`**)**

$$\frac{\begin{array}{l}\mathcal{R}\ \sigma \\ \forall\ \sigma,\ \mathcal{R}\ \sigma \rightarrow \\ \quad [\texttt{while}\ c\ \{t\},\ \sigma \Vdash_L\ \varphi] \wedge \\ \quad [c,\ \sigma \Vdash_R\ \varphi\ \texttt{AU done}_{=}\ (\texttt{true}, \sigma)] \wedge \\ \quad [t,\ \sigma \Vdash_R\ \varphi\ \texttt{AU done}\ (\lambda\ \sigma' \Rightarrow \mathcal{R}\ \sigma')]\end{array}}{[\texttt{while}\ c\ \{t\},\ \sigma \Vdash_L\ \texttt{AG}\ \varphi]}\ \textsc{WhileAG}$$

Coinductive hypothesis $[\texttt{while}\ c\ \{t\},\ \sigma \Vdash_L\ \varphi]$; condition must always be true.

Function Call

Rust: $f(e_1, \ldots, e_n)$

$$\frac{\begin{array}{c}\forall\ j,\ [e_j,\ \sigma_j \Vdash_R\ \varphi\ \texttt{AU done}\ \mathcal{R}_j] \\ \forall\ v_1 \ldots v_n, \sigma',\ \mathcal{R}_1\ v_1\ \sigma_1' \wedge \cdots \wedge \mathcal{R}_n\ v_n\ \sigma' \rightarrow \\ [f(v_1, \ldots, v_n),\ \sigma' \Vdash_R\ \varphi\ \texttt{AU done}\ \mathcal{R}] \\ \forall\ x, \sigma'',\ \mathcal{R}\ x\ \sigma'' \rightarrow [k\ x,\ \sigma'' \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']\end{array}}{[f(e_1, \ldots, e_n),\ \sigma \Vdash_L\ \varphi\ \texttt{AU}\ \varphi']}\ \textsc{CallAU}_L$$

Each e_j evaluated left-to-right maintaining φ; then f called with values.

Single-Process Weakest Preconditions

wp: Assignment

$$\begin{aligned}\texttt{wp}(*x = e,\ \varphi\ \texttt{AU}\ \varphi')\ &=\ \lambda\,\sigma.\ \texttt{wp}(e,\ \varphi\ \texttt{AU done}\ \mathcal{R})\ \sigma \\ &\quad\ \wedge\ \forall\, v, \sigma'.\ \mathcal{R}\ v\ \sigma' \to \texttt{wp}(\texttt{skip},\ \varphi\ \texttt{AU}\ \varphi')\ (\sigma'[x \mapsto v])\end{aligned}$$

wp: Let-binding (Sequential Composition)

$$\begin{aligned}\texttt{wp}(\texttt{let}\ x = t;\ u,\ \varphi\ \texttt{AU}\ \varphi') \ =\ & \lambda\,\sigma.\ \ \texttt{wp}(t,\ \varphi\ \texttt{AU done}\ \mathcal{R})\ \sigma \\ & \wedge\ \ \forall x, \sigma'.\ \mathcal{R}\ x\ \sigma' \to \texttt{wp}(u[x],\ \varphi\ \texttt{AU}\ \varphi')\ \sigma'\end{aligned}$$

wp: Conditional

$$\texttt{wp}(\texttt{if } c\ \{t\}\ \texttt{else}\ \{u\},\ \varphi\ \texttt{AU}\ \varphi') \ = \ \lambda\,\sigma.$$
$$\texttt{wp}(c,\ \varphi\ \texttt{AU done}\ \mathcal{R})\ \sigma\ \wedge\ \forall\ b, \sigma'.\ \mathcal{R}\ b\ \sigma' \rightarrow$$
$$\begin{cases} \texttt{wp}(t,\ \varphi\ \texttt{AU}\ \varphi')\ \sigma', & \text{if } b \\ \texttt{wp}(u,\ \varphi\ \texttt{AU}\ \varphi')\ \sigma', & \text{otherwise} \end{cases}$$

wp: While Loop (AU)

$$
\begin{aligned}
&\texttt{wp}(\texttt{while } c\ \{t\},\ \varphi\ \texttt{AU}\ \varphi') \ =\ \lambda\,\sigma.\\
&\quad \mathcal{R}\ \sigma\ \wedge\ \forall\,\sigma,\ \mathcal{R}\ \sigma \rightarrow\\
&\quad \texttt{wp}(c,\ \varphi\ \texttt{AU done}\ \mathcal{R}')\ \sigma\ \wedge\ \forall\ b, \sigma'.\ \mathcal{R}'\ b\ \sigma' \rightarrow\\
&\quad \begin{cases}
\texttt{wp}(t,\ \varphi\ \texttt{AU}\ \varphi')\ \sigma' \vee & \\
\quad \texttt{wp}(t,\ \varphi\ \texttt{AU done}\ (\lambda\ \_\ \sigma''.\ \mathcal{R}\ \sigma'' \wedge f\ \sigma'' < f\ \sigma))\ \sigma', & \text{if } b\\
\varphi'(\sigma'), & \text{o.w.}
\end{cases}
\end{aligned}
$$

wp: While Loop (AG)

$$
\begin{aligned}
&\texttt{wp}(\texttt{while}\ c\ \{t\},\ \texttt{AG}\ \varphi)\ =\ \lambda\,\sigma.\\
&\quad \mathcal{R}\ \sigma\ \wedge\ \forall\,\sigma,\ \mathcal{R}\ \sigma \rightarrow\\
&\quad \texttt{wp}(\texttt{while}\ c\ \{t\},\ \varphi)\ \sigma\ \wedge\\
&\quad \texttt{wp}(c,\ \varphi\ \texttt{AU}\ \texttt{done}_{=}\ (\texttt{true}, \sigma))\ \sigma\ \wedge\\
&\quad \texttt{wp}(t,\ \varphi\ \texttt{AU}\ \ \texttt{done}\ (\lambda\,\sigma'.\ \mathcal{R}\ \sigma'))\ \sigma
\end{aligned}
$$

Coinductive: $\texttt{wp}(\texttt{while}\ c\ \{t\},\ \varphi)\ \sigma$ is the coinductive hypothesis.

wp: Function Call – f ensures $\texttt{AF done}\ \mathcal{R}_f$

f has terminating postcondition

$$\texttt{wp}(f(e_1, \ldots, e_i),\ \varphi\ \texttt{AU}\ \varphi') \ = \ \lambda\, \sigma.\ \forall\, x, \sigma'.\ \mathcal{R}_f\ x\ \sigma' \to \texttt{wp}(k\ x,\ \varphi\ \texttt{AU}\ \varphi')\ \sigma'$$

where k is the continuation after the call.

f has temporal ensures ψ: transparent checking
Additionally assert $\psi \implies \varphi$ where φ is the caller's obligation.

f ensures $\texttt{AG}\ \psi$: cannot prove $\texttt{AF}$ for it
If f has $\texttt{AG}\ \psi$ (diverges), then $\texttt{wp}(f(\vec{e}),\ \texttt{AF}\ \varphi')$ is **unprovable** – a diverging call can never reach the $\texttt{AF}$ goal.

Multi-Process Configuration

Configuration

A **configuration** $C = (P,\ \sigma,\ i)$:

$P : \texttt{PID} \to \text{Term}$	process dictionary
$\sigma \in \text{State}$	shared mutable state
$i \in \texttt{PID}$	active process

- Temporal formulas φ are over shared state σ
- $\texttt{WP}(C,\ \varphi)$ – multi-process wp, defined by cases on $P(i)$

Cooperative Scheduling

Rust `async` uses **cooperative scheduling**: each process runs uninterrupted until it reaches an `.await`, then yields control to the scheduler.

- **Within a process**: execution is sequential (single-process `wp` applies)
- **At** `.await`: the active process i suspends; the scheduler picks a ready process p to run (nondeterministic choice)
- **No preemption**: a process cannot be interrupted mid-statement; shared state σ is only modified by the active process

This means nondeterminism arises *only* at `.await` boundaries — between yield points, the program is deterministic and the single-process rules apply unchanged.

Multi-Process Weakest Preconditions

WP: Sequential Step

When $P(i)$ is a non-async construct (assignment, let, if, while, sync call):

$$\texttt{WP}((P,\ \sigma,\ i),\ \varphi)\ =\ \texttt{wp}(P(i),\ \varphi)\ \sigma$$

Delegate to single-process `wp`. Process dictionary P is unchanged.

Programs without `async`/`await` reduce entirely to single-process wp:

$$\texttt{WP}((\{0 \mapsto t\},\ \sigma,\ 0),\ \varphi)\ =\ \texttt{wp}(t,\ \varphi)\ \sigma$$

WP: Async (Spawn a Future)

Rust: `async` $\{e\}$ **evaluates to a fresh** $p \in \texttt{PID}$

$$\texttt{WP}((P,\ \sigma,\ i),\ \varphi)\ =\ \texttt{WP}((P[p \mapsto e],\ \sigma,\ i),\ \varphi)$$

- Adds new process p with body e to P (suspended)
- Process i continues running (active, unchanged)
- No state change, no scheduling — futures are lazy

WP: Await (AU – terminating callee)

Rust: $P(i) = \texttt{let}\ x = p.\texttt{await};\ k\ x$

$$\begin{aligned}
&\texttt{WP}((P,\ \sigma,\ i),\ \varphi\ \texttt{AU}\ \varphi') \ = \\
&\quad \texttt{WP}((P,\ \sigma,\ p),\ \varphi\ \texttt{AU done}\ \mathcal{R}) \\
&\quad \wedge\ \forall\ x, \sigma'.\ \mathcal{R}\ x\ \sigma' \to \texttt{WP}((P[i \mapsto k[x]],\ \sigma',\ i),\ \varphi\ \texttt{AU}\ \varphi')
\end{aligned}$$

- Switch active process to p; run it maintaining φ
- After p terminates with result x and state σ', resume i

WP: Await (AG – diverging callee)

Rust: $P(i) = p.\texttt{await}$

$$\texttt{WP}((P,\ \sigma,\ i),\ \texttt{AG}\ \varphi)\ =\ \texttt{wp}(P(p),\ \texttt{AG}\ \varphi)\ \sigma$$

- Reduces to single-process `wp` on callee's body $P(p)$
- Process p must satisfy `AG` φ (runs forever)
- Continuation after await is unreachable

Vera Implementation

Vera Syntax

Vera extends Verus with temporal postconditions and loop annotations:

Annotation	TICL	Proof obligation
`ensures af(done(Q))`	AF done Q	Loop + `decreases`
`ensures ag(P)`	AG φ	Infinite loop + `invariant`
`ensures au(P, done(Q))`	P AU done Q	Loop + `invariant` + `decreases`
`ensures ag(af(now(Q)))`	AG AF now Q	Ghost accumulator + `decreases`
`invariant R`	$\mathcal{R}$	Temporal refinement mapping
`decreases f`	f	Well-founded metric (progress)

`done(Q)`: Q holds at termination. `now(Q)`: Q holds at some intermediate state.

Example: Queue Drain (AF)

Liveness: the queue eventually becomes empty

```
fn drain(q: &mut Queue)
    requires q.view().len() > 0,
    ensures af(done(q.view().len() == 0)),
{
    while q.len() > 0
        invariant q.view().len() >= 0,
        decreases q.view().len(),
    {
        let _ = q.dequeue();
    }
}
```

$\texttt{invariant} = \mathcal{R}, \quad \texttt{decreases} = f \quad \Rightarrow \quad \text{WhileAU}_L \text{ rule applies.}$

Example: Bounded Counter (AG)

Safety: the counter stays bounded forever

```
fn bounded_counter(x: &mut u64)
    requires *x == 0,
    ensures ag(*x <= 15),
{
    loop
        invariant *x <= 10,
    {
        if *x < 10 { *x = *x + 1; }
        else       { *x = 0; }
    }
}
```

No `decreases` $\Rightarrow$ infinite loop $\Rightarrow$ WhileAG rule.

`invariant` $(*x \leq 10) \implies$ postcondition $(*x \leq 15)$.

Example: Bind Rule (AG + AF callee)

AG caller invokes AF callee — bind rule extracts postcondition

```
fn reset_to_zero(x: &mut u64)
    ensures af(done(*x == 0)),
{ *x = 0; }

fn ag_caller(x: &mut u64)
    requires *x == 0,
    ensures ag(*x <= 10),
{
    loop
        invariant *x <= 10,
    {
        if *x < 10 { *x = *x + 1; }
        if *x > 5 { reset_to_zero(x); }
    }
}
```

After `reset_to_zero`: assume $*x = 0$ (bind rule $\mathcal{R}$). Invariant preserved.

Example: Round-Robin Fairness (AG(AF(now)))

Every element eventually reaches the head of the queue

```
fn round_robin(queue: &mut Queue, Ghost(x): Ghost<u64>)
    requires
        queue.view().len() > 1,
        queue.view().contains(x),
    ensures
        ag(queue.view().len() > 0),
        ag(af(now(queue.peek_spec() == x))),
{
    loop
        invariant
            queue.view().len() > 1,
            queue.view().contains(x),
        decreases queue.view().index_of_first(x),
    {
        let val = queue.dequeue();
        queue.enqueue(val);
    }
}
```

now() = state predicate (head = x at loop start, not end). Ghost accumulator tracks it.

Summary

Complete Rule Set

Construct	Rule	Layer
$*x = e$	State update	wp
let $x = t$; u	BindAU$_L$	wp
if c $\{t\}$ else $\{u\}$	Case split (effectful c)	wp
while c $\{t\}$ + dec.	WhileAU$_L$	wp
while c $\{t\}$ (no exit)	WhileAG	wp
$f(\vec{e})$	Contract-dependent	wp
Sequential step	Delegate to wp	WP
async $\{e\}$	Extend P, no-op	WP
p.await (AU callee)	Switch active to p	WP
p.await (AG callee)	Divergence	WP

Two layers: $\mathtt{wp}(t,\ \varphi)$ for sequential Rust, $\mathtt{WP}((P,\ \sigma,\ i),\ \varphi)$ for async concurrency.